DESENVOLVIMENTO DE APLICATIVOS MOBILE · MÓDULO 00

Apresentação e Ecossistema Mobile

O que é desenvolvimento mobile, plataformas, arquitetura do Android e a stack do curso

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o que diferencia o desenvolvimento mobile do desenvolvimento web e desktop.
- Comparar as abordagens **nativa**, **multiplataforma**, **híbrida** e **PWA**.
- Descrever a arquitetura em camadas do Android e os componentes de um app.
- Entender `minSdk`, `targetSdk` e `compileSdk`.
- Justificar a stack adotada no curso: Kotlin + Jetpack Compose.

Pré-requisitos

Lógica de programação e noções de orientação a objetos. Experiência com Java, C, C++ ou Python ajuda, mas não é obrigatória.

Roteiro

Neste módulo

- Sobre o curso
- O que é desenvolvimento mobile
- Plataformas: Android e iOS
- Arquitetura do Android
- A stack do curso
- Fechamento

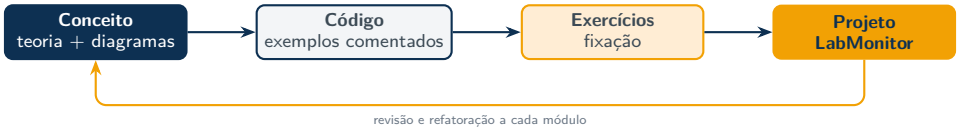
Sobre o curso

Organização em módulos

00 Apresentação e ecossistema mobile
01 Ambiente de desenvolvimento
02 Kotlin essencial
03 Kotlin assíncrono: coroutines e Flow
04 Interfaces com Jetpack Compose
05 Estado e navegação
06 Arquitetura de apps
07 Persistência local
08 Rede e consumo de APIs
09 Firebase: backend como serviço
10 Recursos do dispositivo e IoT
11 Testes e qualidade
12 Publicação na Google Play
13 Desenvolvimento multiplataforma
14 Tópicos avançados e projeto final

Cada módulo é um conjunto de slides independente, com teoria, código, exercícios e uma etapa do projeto integrador.

Metodologia de estudo



Como estudar

- Digite o código, não copie e cole.
- Rode tudo no emulador *e* num celular real.
- Faça um commit no Git ao fim de cada etapa.

Carga sugerida

- 6 a 10 horas por módulo.
- Os módulos 02 a 06 são a base de tudo: não pule.

Projeto integrador: *LabMonitor*



Um app para **laboratórios de eletroeletrônica**, construído ao longo do curso:

- cadastro de equipamentos e bancadas (**M04–M07**);
- reservas e agenda sincronizadas na nuvem (**M08–M09**);
- leitura de sensores de um **ESP32** via BLE/MQTT (**M10**);
- testes automatizados (**M11**) e publicação na Play Store (**M12**);
- detecção de anomalias com IA no aparelho (**M14**).

Por que um projeto integrador?

Cada conceito novo é aplicado no mesmo código, como acontece num produto real que evolui.

O que é desenvolvimento mobile

O ambiente móvel é diferente

- **Recursos limitados:** bateria, memória e CPU são compartilhados com dezenas de apps.
- **Rede intermitente:** 4G/5G, Wi-Fi e modo avião; o app precisa funcionar *offline*.
- **Telas diversas:** de relógios a tablets e dobráveis, em retrato e paisagem.
- **Interrupções:** chamadas, notificações e troca de app a qualquer momento.
- **Ciclo de vida controlado pelo sistema:** o SO pode encerrar o processo para liberar memória.
- **Sensores e hardware:** câmera, GPS, acelerômetro, Bluetooth, NFC.
- **Privacidade:** permissões explícitas e sandbox por app.
- **Distribuição por lojas:** revisão, políticas e atualizações graduais.

Consequência prática

Um bom app mobile é projetado para **ser interrompido**, **perder a rede** e **ser destruído** sem perder dados do usuário.

Abordagens de desenvolvimento

Abordagem	Como funciona	Vantagens	Limitações
Nativa	Kotlin (Android) ou Swift (iOS), usando o SDK oficial	Desempenho máximo, acesso total a APIs e hardware	Dois códigos para duas plataformas
Multiplataforma	Flutter, React Native, Kotlin Multiplatform	Um código (ou lógica) para Android e iOS	Camada extra; recursos novos chegam depois
Híbrida	App web dentro de uma *WebView* (Ionic, Cordova)	Reaproveita HTML/CSS/JS	Desempenho e aparência inferiores
PWA	Site instalável com *service workers*	Sem loja, atualização imediata	Acesso limitado ao hardware, sobretudo no iOS

Dica

Não existe abordagem "melhor". Existe a mais adequada ao **público**, ao **orçamento**, à **equipe** e aos **recursos de hardware** exigidos.

Linha do tempo resumida



Tendência

O desenvolvimento Android migrou de **Java + XML** (imperativo) para **Kotlin + Compose** (declarativo). Materiais antigos ainda usam o modelo anterior; saiba reconhecê-lo, mas comece pelo atual.

Plataformas: Android e iOS

Android *versus* iOS

	Android	iOS
Linguagem oficial	Kotlin (Java ainda suportado)	Swift (Objective-C legado)
UI moderna	Jetpack Compose	SwiftUI
IDE	Android Studio (Windows, Linux, macOS)	Xcode (somente macOS)
Loja	Google Play (e lojas alternativas)	App Store
Conta de desenvolvedor	Taxa única	Anuidade
Dispositivos	Milhares de modelos e fabricantes	Poucos modelos, da Apple
Desafio típico	Fragmentação de versões e telas	Revisão rigorosa e ecossistema fechado

No Brasil

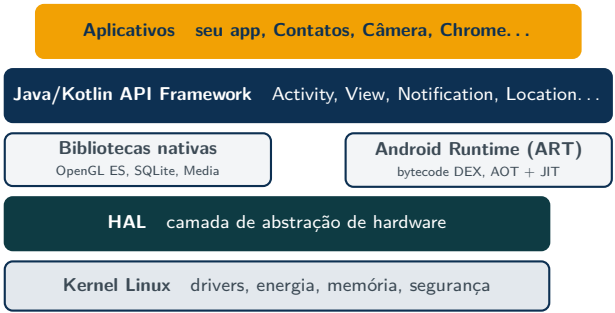
O Android está na grande maioria dos smartphones em uso. Por isso o curso começa por ele.

iOS sem Mac?

Com Flutter ou Kotlin Multiplatform é possível escrever o código, mas compilar e publicar para iOS exige macOS/Xcode (ou um serviço de CI na nuvem).

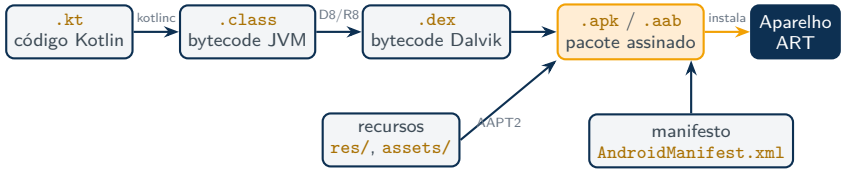
Arquitetura do Android

A pilha de software do Android



- O **kernel Linux** gerencia processos, memória e drivers.
- A **HAL** padroniza o acesso a câmera, sensores e Bluetooth entre fabricantes.
- O **ART** executa o bytecode `.dex` gerado a partir do Kotlin.
- Seu app conversa quase sempre só com o **framework**.

Do código-fonte ao aparelho



APK

Android Package: arquivo instalável com código, recursos e manifesto. É o que roda no aparelho.

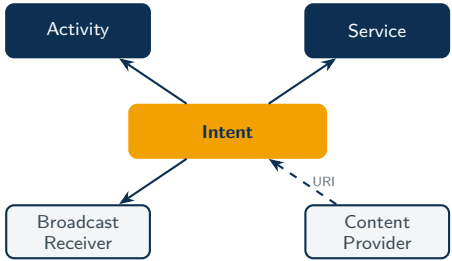
AAB

Android App Bundle: formato de **publicação** exigido pela Google Play, que gera APKs otimizados para cada aparelho.

O **Gradle** orquestra todas essas etapas (Módulo 01).

Componentes fundamentais de um app

- **Activity**: uma tela/janela com a qual o usuário interage. Com Compose, um app costuma ter **uma única** Activity.
- **Service**: trabalho sem interface (ex.: tocar música, sincronizar dados).
- **BroadcastReceiver**: reage a eventos do sistema (bateria fraca, boot concluído).
- **ContentProvider**: expõe dados a outros apps (ex.: contatos).
- **Intent**: mensagem que ativa componentes, do seu app ou de outros.



Dica

Todos os componentes são declarados no `AndroidManifest.xml`, junto com as permissões que o app pede.

Níveis de API `minSdk`, `targetSdk` e `compileSdk`

- **API level**: número inteiro que identifica cada versão do Android (Android 16 = API 36).
- `minSdk`: versão **mínima** em que o app instala.
- `targetSdk`: versão para a qual o app foi **testado**; ativa os comportamentos novos do sistema.
- `compileSdk`: versão das APIs usadas na **compilação**.

app/build.gradle.kts

```
android {
  compileSdk = 36
  defaultConfig {
    minSdk = 26      // Android 8.0
    targetSdk = 36   // Android 16
  }
}
```

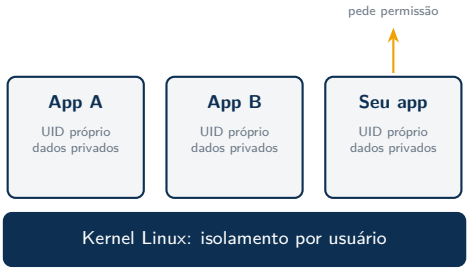
Exigência da Google Play

Desde **31/08/2026**, apps novos e atualizações enviados à Google Play precisam ter `targetSdk` igual ou superior a **36** (Android 16). Esse requisito sobe todo ano.

Dica

Um `minSdk` entre 24 e 26 cobre praticamente todos os aparelhos ativos e simplifica muito o código.

Segurança: sandbox e permissões



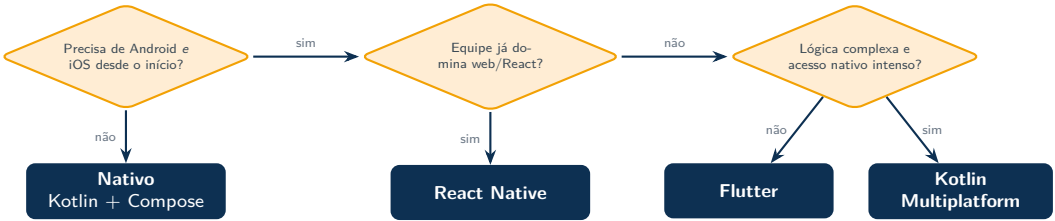
- Cada app roda em seu próprio processo, com um usuário Linux exclusivo.
- Um app não lê os arquivos privados de outro.
- Permissões **normais** (internet) são concedidas na instalação.
- Permissões **perigosas** (câmera, localização) são pedidas **em tempo de execução** e o usuário pode negar.

Verificação de desenvolvedor no Brasil

Desde **30/09/2026**, aparelhos Android certificados no Brasil só instalam normalmente apps registrados por **desenvolvedores verificados**, mesmo fora da Play Store. Durante o desenvolvimento a instalação via **ADB** continua funcionando (Módulos 01 e 12).

A stack do curso

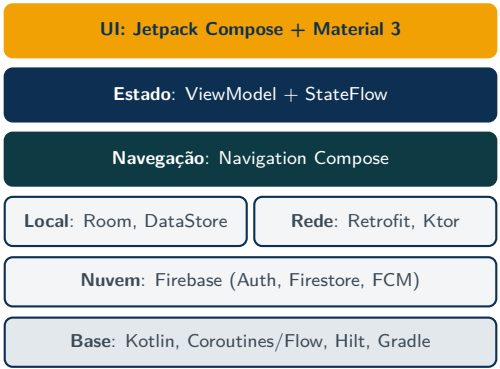
Como escolher a tecnologia



Dica

Aprender Kotlin + Compose primeiro facilita a migração para Kotlin Multiplatform e torna Flutter e React Native mais fáceis de entender (Módulo 13).

A stack adotada



- É a stack que o Google recomenda e documenta oficialmente.
- É a mais pedida em vagas Android.
- A arquitetura (MVVM, repositório, injeção de dependência) vale para **qualquer** plataforma.

Jetpack

Conjunto de bibliotecas oficiais (`androidx.*`) que resolvem problemas comuns com compatibilidade entre versões do Android.

Fechamento

Resumo

Resumo

- O ambiente móvel impõe restrições de energia, rede, tela e ciclo de vida.
- Há quatro abordagens: nativa, multiplataforma, híbrida e PWA, cada uma com seus trade-offs.
- O Android é uma pilha em camadas: kernel Linux, HAL, ART, framework e apps.
- Os componentes de um app são Activity, Service, BroadcastReceiver e ContentProvider, ligados por Intents.
- `targetSdk` precisa acompanhar a exigência anual da Google Play (API 36 em 2026).
- O curso usa Kotlin + Jetpack Compose + MVVM + Room + Retrofit + Firebase.

Exercícios

Exercício 1 — Pesquisa

Escolha três apps que você usa todos os dias e tente identificar a abordagem de cada um (nativa, multiplataforma, híbrida ou PWA). Que pistas você usou?

Exercício 2 — Análise

Para cada cenário, indique e justifique uma abordagem: (a) app de lanterna; (b) app de um evento acadêmico com duração de uma semana; (c) app que lê um sensor BLE a 100 Hz; (d) startup que precisa lançar em Android e iOS em três meses.

Exercício 3 — Conceitos

Explique com suas palavras a diferença entre `minSdk` e `targetSdk`. O que acontece se o `targetSdk` ficar desatualizado?

LabMonitor — Etapa 0

Escreva em uma página: quem usa o app, quais as 5 funcionalidades principais e quais recursos de hardware ele usa.

Para aprofundar

- **Android Developers** — Visão geral da plataforma: `https://developer.android.com/guide/platform`
- **Fundamentos de apps**: `https://developer.android.com/guide/components/fundamentals`
- **Requisitos de API de destino da Google Play**: `https://developer.android.com/google/play/requirements/target-sdk`
- **Verificação de desenvolvedor Android**: `https://developer.android.com/developer-verification`
- **Curso oficial gratuito**: Android Basics with Compose — `https://developer.android.com/courses/android-basics-compose/course`

Fim do Módulo 00

Próximo: Módulo 01 — Ambiente de desenvolvimento: Android Studio, emulador, Gradle e o primeiro app.